PROJECT REPORT

On

“Student Attendance Tracker”

(Mark Attendance and Show Report)

[**Experiential Learning**]

Submitted By:

Tanmay Niwate [CM25058]

Guided By:

Ms. Suchita meshkar



DEPARTMENT OF CSE (AI &ML)

S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH, NAGPUR.

(An Autonomous Institute, Affiliated to RTMNU, Nagpur)

2026-2027

© S.B.J.I.T.M.R Nagpur

INDEX

INDEX		i
CHAPTER 1	INTRODUCTION	1-2
CHAPTER 2	METHODOLOGY	3-4
CHAPTER 3	TOOLS/PLATFORMS	5
CHAPTER 4	DESIGN & IMPLEMENTATION	6-11
	4.1 ALGORITHM	
	4.2 FLOWCHART	
	4.3 SOURCE CODE	
CHAPTER 5	RESULT & DISCUSSION	12-16
	5.1 OUTPUT	
	5.2 DISCUSSION	
	5.3 APPLICATION	
CHAPTER 6	CONCLUSION	17
	REFERENCES	18

CHAPTER 1
INTRODUCTION

Attendance is the record of the lectures which a student has attended. In every college a minimum attendance, such as 75%, is required to be eligible for the examination. In many classes the teacher still marks the attendance on paper and calculates the percentage by hand at the end of the month or the semester. This takes time and the calculation may also go wrong.

In this project a **Student Attendance Tracker** is made in Python for the course Python Programming Language (N-PCCCM304P). The teacher enters the students and the number of lectures, marks every student Present (P) or Absent (A) for every lecture, and the program then shows a complete attendance report. The report has the total lectures, lectures present, lectures absent, the attendance percentage and the status of every student. A student whose attendance is below 75% is shown as DEFAULTER and the others are shown as REGULAR.

The project looks simple but it is useful for a beginner because many basic concepts of Python are used in it, such as variables, input and output, conditional statements, loops, lists, dictionaries, tuples and functions.

PROBLEM STATEMENT

To develop a Python based application which can mark the attendance of students lecture by lecture, calculate the attendance percentage of every student, find the students whose attendance is below the minimum limit of 75% (defaulters) and display a clear attendance report. The data is managed only with variables and data structures, so no file handling or database is required.

BACKGROUND

The project was completed in different phases. In Phase I the problem statement, objectives, algorithm, flowchart and the Python logic were prepared. In Phase II this design was converted into a working Python program. After that an advanced version was made which has input validation, an automated test suite, a summary of the class and a list of defaulters. In Phase III a desktop GUI (Tkinter) was made with the tabs Dashboard, Students, Mark Attendance and Report, so that the attendance can be marked and seen with a few clicks.

OBJECTIVES

- To develop a Python based student attendance tracking system.
- To mark the attendance of every student as Present or Absent for every lecture.
- To calculate the attendance percentage of every student automatically.
- To identify the students below 75% as Defaulters and the others as Regular.

- To show a clear and formatted attendance report with a class summary.
- To make attendance monitoring faster, easier and more accurate.

FEATURES

- Mark attendance (P / A) for each student in each lecture.
- Input validation: the program accepts p, P, present, a, A, absent. For a wrong input like "x" or an empty input it asks again instead of crashing. The number of students and lectures must be greater than zero.
- Attendance percentage and status (REGULAR / DEFAULTER) are calculated automatically with the 75% rule.
- Class summary: class average, highest attendance, lowest attendance, number of regular students and defaulters.
- A list of defaulter students is displayed at the end.
- A desktop GUI (Tkinter) with the tabs Dashboard, Students, Mark Attendance and Report. Students can be added or deleted, and the status of a student (Present, Absent or Late) is changed by a double-click.
- An automated test suite of 18 assert based tests runs every time before the program starts.
- A demo mode (`--demo`) which marks the attendance of 8 sample students for 20 lectures automatically.

CHAPTER 2
METHODOLOGY

This chapter explains, step by step, the process which was followed to develop the project from the beginning to the end.

Step 1: Understanding the problem

First the working of attendance in a class was studied. The program has to take the students and the lectures, mark every student present or absent, calculate the percentage, find the defaulters and show the report.

Step 2: Planning the solution

Before writing any code, the steps of the program were written as an algorithm in simple English. After that a flowchart and a pseudocode were prepared. This helped to understand the flow of the program (input, processing, output and the repetition) clearly.

Step 3: Selecting the Python concepts

The concepts needed were decided: variables to store counts and percentage, input() and print() for input and output, a list of tuples for the students, a dictionary to keep the P / A marks of every student, if-else statements for the 75% rule, for and while loops for lectures and students, and functions to divide the program into small reusable parts.

Step 4: Coding (Phase II)

The algorithm was converted into Python code. The students were stored as (roll no., name) pairs, the marks of every student were stored in a dictionary of lists, and the functions calculate_percentage() and get_status() were written for the calculation and the 75% rule.

Step 5: Input validation

The teacher may type the mark in capital letters, with extra spaces, or may type something wrong. Therefore the input is first cleaned (spaces removed and converted to lower case) by the function normalize_mark() and then matched with the valid values. If it does not match, the teacher is asked to enter the mark again. In the same way the number of students and lectures must be a number greater than zero.

Step 6: Testing

An assert based test suite was written in the function run_test_cases(). It checks the percentage calculation, the status for values below, equal to and above 75%, and 7 different raw inputs. The tests run automatically each time the program is started, so any mistake in the logic is found before the program begins.

Step 7: Report and analysis

After the attendance is marked, the function build_report() makes the student-wise report and the function analyze() calculates the class average, highest, lowest, number of regular students and defaulters, which are then displayed.

Step 8: Demo mode

To test the full program without typing 160 marks by hand, a demo mode was added. In this mode the attendance of 8 sample students is generated automatically for 20 lectures with a fixed random seed, so the same output is obtained every time.

Step 9: Phase III desktop GUI

In the last phase a GUI based version was made with Tkinter. It has four tabs: Dashboard, Students, Mark Attendance and Report. The teacher selects a date, changes the status of every student by double-click and sees the report with the attendance percentage and status.

Step 10: Documentation

The source code, test results and screenshots were collected, and this report was prepared to explain the work.

CHAPTER 3
TOOLS/PLATFORMS

3.1 SOFTWARE REQUIREMENT

Requirement	Details
Programming language	Python 3
Built-in modules	random (sample attendance in demo mode), sys (demo mode option)
GUI library	tkinter (Phase III desktop interface, comes with Python)
Operating system	Windows, Linux or macOS
Editor / IDE	Any Python editor such as IDLE or VS Code
Version control	Git and GitHub (to store the source code)

Python

Python is an easy to learn and widely used programming language. Its simple syntax makes it a good choice for a beginner level project like this one.

Data structures

A list of tuples stores the roll number and name of the students. A dictionary stores the list of P / A marks of every student against the roll number. Because everything is kept in these variables, no file or database is needed.

random module

This module is used only in the demo mode. The function random.seed() is used with a fixed number so that the same sample attendance is generated on every run.

tkinter

tkinter is the standard GUI library of Python. It is used in Phase III to make the desktop window with the Dashboard, Students, Mark Attendance and Report tabs, buttons and tables.

Git and GitHub

Git is used to keep track of the changes in the code and GitHub can be used to store the project online.

CHAPTER 4
DESIGN & IMPLEMENTATION

4.1 ALGORITHM

1. Start the program.
2. Enter the number of students and the name of every student (roll numbers are given automatically).
3. Enter the number of lectures conducted.
4. For every lecture, mark every student as Present (P) or Absent (A). If the input is wrong, ask again.
5. Count the lectures present for every student.
6. Calculate the attendance percentage = (lectures present / total lectures) x 100.
7. Compare the percentage with 75%.
8. If the percentage is less than 75%, mark the student as DEFAULTER, otherwise mark the student as REGULAR.
9. Display the attendance report of all the students.
10. Display the class summary and the list of defaulters.
11. Stop the program.

Rule used in the algorithm: 75% and above is REGULAR (eligible). Below 75% is DEFAULTER. A student having exactly 75% is REGULAR.

4.2 FLOWCHART

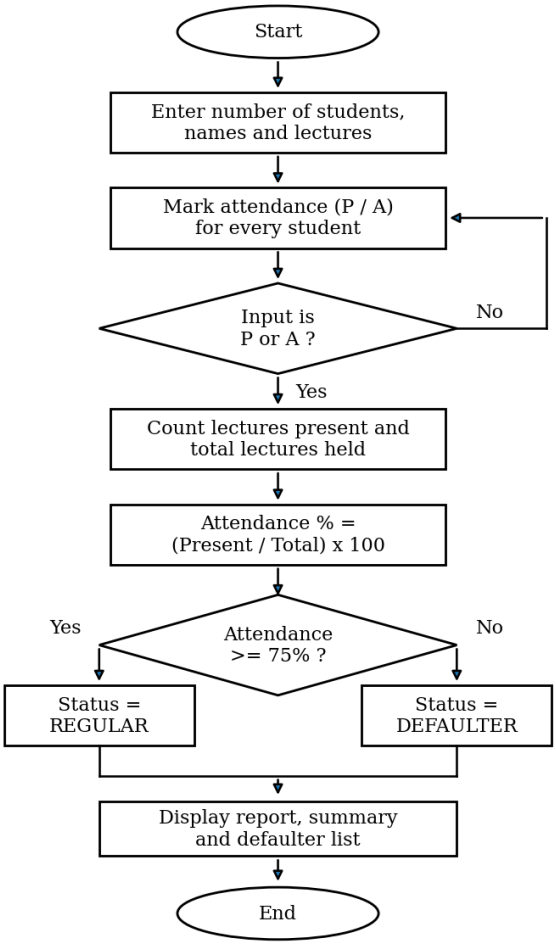


Fig. 4.2.1 Flowchart of Student Attendance Tracker

PSEUDOCODE

```
START
  INPUT number of students, names
  INPUT total lectures
  FOR lecture = 1 TO total lectures
    FOR each student
      INPUT mark (P / A) until valid
      STORE mark in the record of the student
    END FOR
  END FOR
  FOR each student
    present = COUNT of 'P' in the record
    percentage = (present / total lectures) * 100
    IF percentage < 75
      status = DEFAULTER
    ELSE
      status = REGULAR
    END IF
  END FOR
  DISPLAY attendance report
  DISPLAY class summary and defaulter list
END
```

4.3 SOURCE CODE

The complete program is given below (file name: attendance_tracker.py). It is run normally for entering the attendance by hand and with the option `--demo` for the automatic demo with 8 students and 20 lectures.

```
"""Student Attendance Tracker - Mark Attendance and Show Report (console version)."""
import random
import sys

THRESHOLD = 75.0          # minimum attendance percentage
DEMO_SEED = 2026
DEMO_LECTURES = 20

DEMO_STUDENTS = [
    (1, "Aarav Deshmukh"), (2, "Isha Kulkarni"), (3, "Rohan Wankhede"),
    (4, "Sneha Thakre"), (5, "Vedant Raut"), (6, "Pooja Meshram"),
    (7, "Karan Bhise"), (8, "Nidhi Gawande"),
]
# chance of being present for each demo student (used only in demo mode)
RATES = {1: 0.95, 2: 0.90, 3: 0.70, 4: 0.85, 5: 0.55, 6: 0.80, 7: 0.65, 8: 0.92}


def calculate_percentage(present, total):
    """Attendance % = (present / total) * 100. Returns 0.0 if no lecture held."""
    if total <= 0:
        return 0.0
    return present / total * 100


def get_status(percentage):
    """REGULAR if percentage >= THRESHOLD, else DEFAULTER."""
    return "REGULAR" if percentage >= THRESHOLD else "DEFAULTER"


def normalize_mark(raw):
    """Clean the input. Returns 'P', 'A' or None for invalid input."""
    text = raw.strip().lower()
    if text in ("p", "present", "1"):
        return "P"
    if text in ("a", "absent", "0"):
        return "A"
    return None


def run_test_cases():
    """Assert based tests, executed every time before the program starts."""
    pct_cases = [
        (0, 20, 0.0), (10, 20, 50.0), (15, 20, 75.0),
        (20, 20, 100.0), (14, 20, 70.0), (0, 0, 0.0),
    ]
    for present, total, expected in pct_cases:
        assert calculate_percentage(present, total) == expected, (present, total)

    status_cases = [
        (100.0, "REGULAR"), (75.0, "REGULAR"), (74.9, "DEFAULTER"),
        (50.0, "DEFAULTER"), (0.0, "DEFAULTER"),
    ]
    for pct, expected in status_cases:
        assert get_status(pct) == expected, pct

    mark_cases = [
        ("p", "P"), ("P", "P"), (" present ", "P"), ("a", "A"),
        ("ABSENT", "A"), ("x", None), ("", None),
    ]
```

```
    for raw, expected in mark_cases:
        assert normalize_mark(raw) == expected, raw

    total = len(pct_cases) + len(status_cases) + len(mark_cases)
    print(f"Test suite: {total} / {total} automated tests passed.")


def get_mark(name):
    while True:
        mark = normalize_mark(input(f"  {name} (P/A): "))
        if mark is not None:
            return mark
        print("  Invalid input. Please enter P or A.")


def get_positive_int(prompt):
    while True:
        text = input(prompt).strip()
        if text.isdigit() and int(text) > 0:
            return int(text)
        print("Invalid input. Please enter a number greater than zero.")


def enter_students():
    students = []
    count = get_positive_int("Enter number of students: ")
    for i in range(1, count + 1):
        name = input(f"Enter name of student {i}: ").strip()
        while name == "":
            name = input("Name cannot be empty. Enter again: ").strip()
        students.append((i, name))
    return students


def mark_attendance(students, lectures, demo=False):
    """records[roll] = list of 'P'/'A', one entry per lecture."""
    records = {roll: [] for roll, _ in students}
    for lec in range(1, lectures + 1):
        if not demo:
            print(f"\nLecture {lec}: mark attendance")
        for roll, name in students:
            if demo:
                mark = "P" if random.random() < RATES[roll] else "A"
            else:
                mark = get_mark(f"{roll}. {name}")
            records[roll].append(mark)
    return records


def build_report(students, records):
    report = []
    for roll, name in students:
        total = len(records[roll])
        present = records[roll].count("P")
        pct = calculate_percentage(present, total)
        report.append({"roll": roll, "name": name, "total": total,
                       "present": present, "absent": total - present,
                       "pct": pct, "status": get_status(pct)})
    return report


def analyze(report):
    pcts = [r["pct"] for r in report]
    defaulters = [r for r in report if r["status"] == "DEFAULTER"]
    return {
        "students": len(report),
        "average": sum(pcts) / len(pcts),
        "highest": max(report, key=lambda r: r["pct"]),
```

```
        "lowest": min(report, key=lambda r: r["pct"]),
        "defaulters": len(defaulters),
        "regular": len(report) - len(defaulters),
    }


def print_report(report):
    line = "-" * 66
    print("\nATTENDANCE REPORT")
    print(line)
    print(f"{'Roll':<5}{'Name':<20}{'Total':>6}{'Present':>9}{'Absent':>8}{'%':>8}  Status")
    print(line)
    for r in report:
        print(f"{r['roll']:<5}{r['name']:<20}{r['total']:>6}{r['present']:>9}"
              f"{r['absent']:>8}{r['pct']:>7.1f}%  {r['status']}")
    print(line)


def main():
    run_test_cases()
    demo = "--demo" in sys.argv
    if demo:
        random.seed(DEMO_SEED)
        students, lectures = DEMO_STUDENTS, DEMO_LECTURES
        print(f"Demo mode: {len(students)} students, {lectures} lectures.")
    else:
        students = enter_students()
        lectures = get_positive_int("Enter number of lectures conducted: ")

    records = mark_attendance(students, lectures, demo)
    report = build_report(students, records)
    print_report(report)

    stats = analyze(report)
    print("\nSUMMARY")
    print(f"Total students     : {stats['students']}")
    print(f"Class average      : {stats['average']:.1f}%")
    print(f"Highest attendance : {stats['highest']['name']} ({stats['highest']['pct']:.1f}%)")
    print(f"Lowest attendance  : {stats['lowest']['name']} ({stats['lowest']['pct']:.1f}%)")
    print(f"Regular students   : {stats['regular']}")
    print(f"Defaulter students : {stats['defaulters']}")

    if stats["defaulters"]:
        print("\nDEFAULTER LIST (below 75%):")
        for r in report:
            if r["status"] == "DEFAULTER":
                print(f"  {r['roll']}. {r['name']} - {r['pct']:.1f}%")


if __name__ == "__main__":
    main()
```

Fig. 4.3.1 Source code of Student Attendance Tracker (attendance_tracker.py)

CHAPTER 5
RESULT & DISCUSSION

5.1 OUTPUT

The program was run in demo mode (8 sample students, 20 lectures). The names and the attendance in the demo are sample data. The following figures show the output.

```
Test suite: 18 / 18 automated tests passed.
Demo mode: 8 students, 20 lectures.
```

Fig. 5.1.1 Test suite result and start of the demo run

```
ATTENDANCE REPORT
---------------------------------------------------------------------
Roll Name                 Total  Present  Absent       %  Status
---------------------------------------------------------------------
1    Aarav Deshmukh          20       19       1   95.0%  REGULAR
2    Isha Kulkarni           20       15       5   75.0%  REGULAR
3    Rohan Wankhede          20       14       6   70.0%  DEFAULTER
4    Sneha Thakre            20       16       4   80.0%  REGULAR
5    Vedant Raut             20       10      10   50.0%  DEFAULTER
6    Pooja Meshram           20       18       2   90.0%  REGULAR
7    Karan Bhise             20       12       8   60.0%  DEFAULTER
8    Nidhi Gawande           20       17       3   85.0%  REGULAR
---------------------------------------------------------------------
```

Fig. 5.1.2 Attendance report of the demo run

```
SUMMARY
Total students     : 8
Class average      : 75.6%
Highest attendance : Aarav Deshmukh (95.0%)
Lowest attendance  : Vedant Raut (50.0%)
Regular students   : 5
Defaulter students : 3

DEFAULTER LIST (below 75%):
  3. Rohan Wankhede - 70.0%
  5. Vedant Raut - 50.0%
  7. Karan Bhise - 60.0%
```

Fig. 5.1.3 Class summary and defaulter list

The Phase III desktop GUI version of the tracker is shown in the following figures.

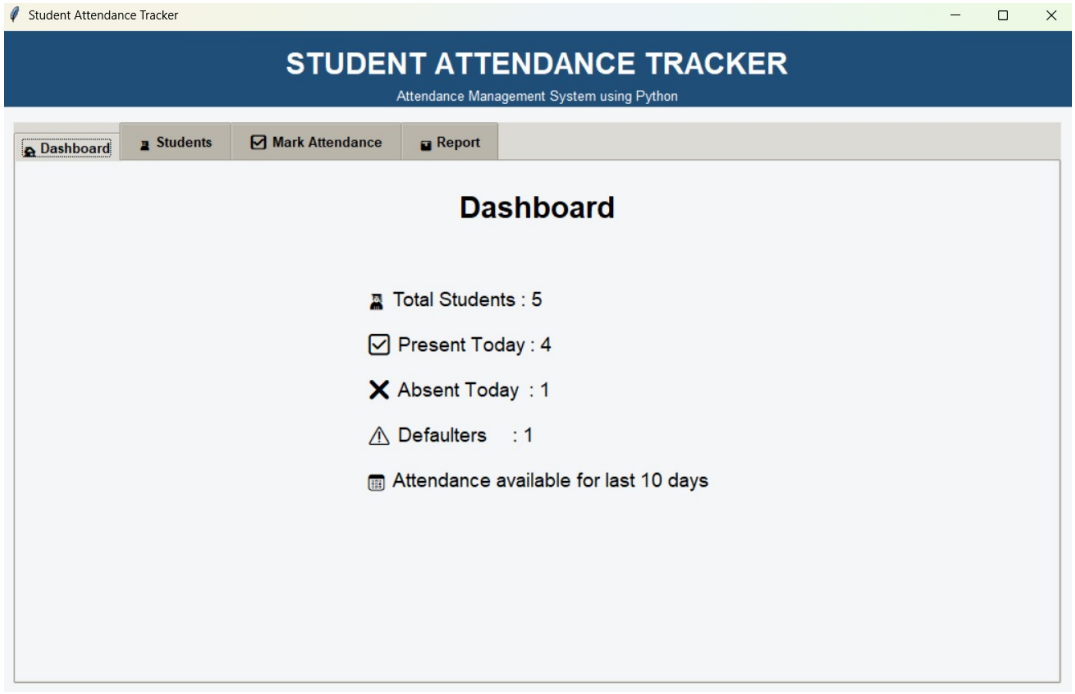


Fig. 5.1.4 Dashboard tab of the GUI

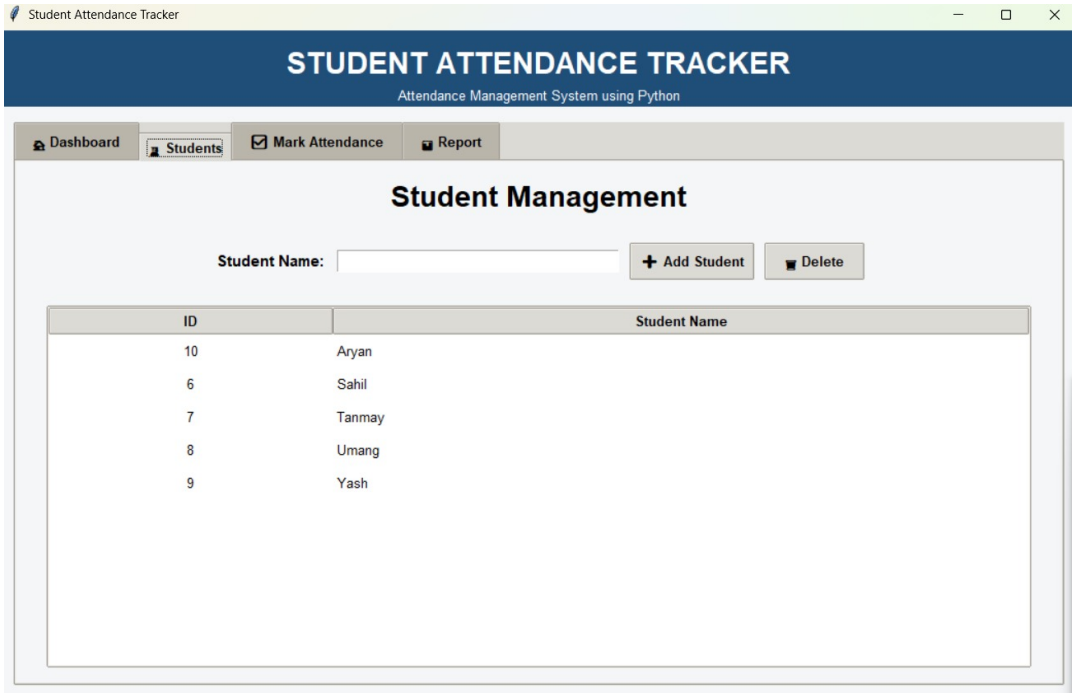


Fig. 5.1.5 Students tab (student management)

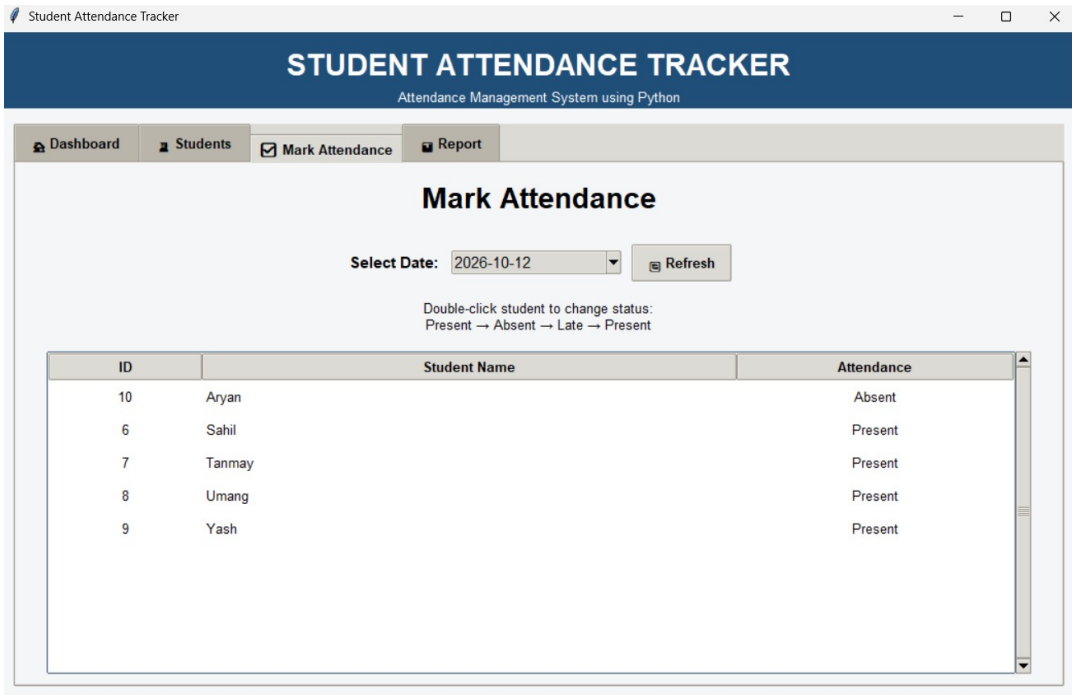


Fig. 5.1.6 Mark Attendance tab

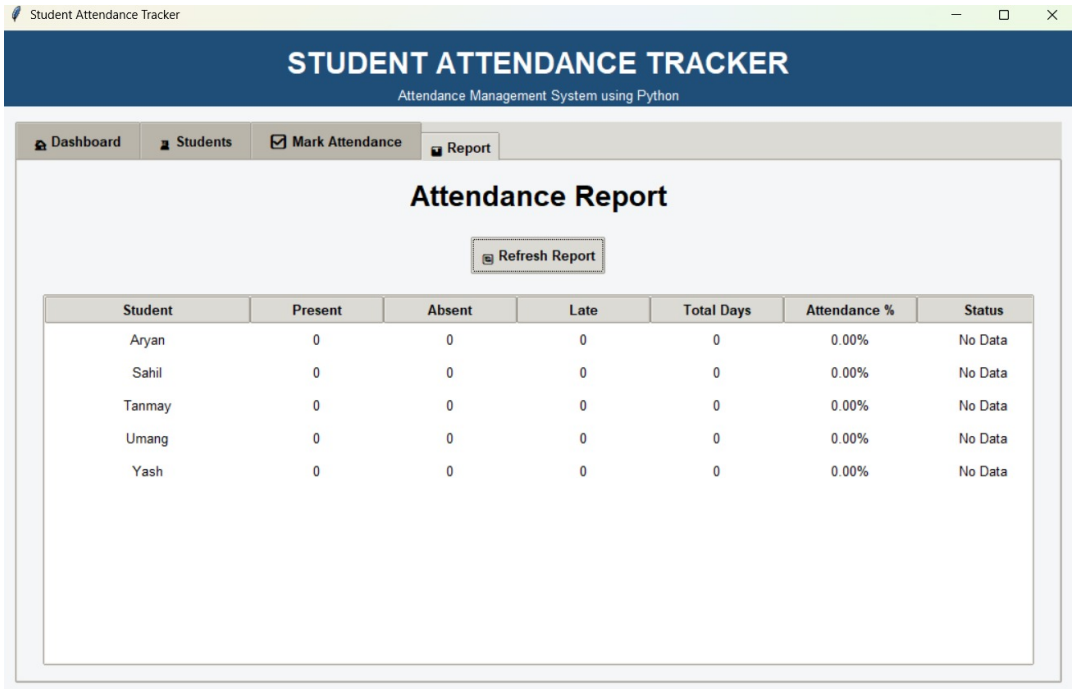


Fig. 5.1.7 Report tab (attendance report)

5.2 DISCUSSION

The demo run gave the following statistics, which are calculated by the function analyze().

Metric	Value
Total students	8
Total lectures	20
Class average attendance	75.6%
Highest attendance	Aarav Deshmukh (95.0%)
Lowest attendance	Vedant Raut (50.0%)
Regular students	5
Defaulter students	3 (Rohan Wankhede, Vedant Raut, Karan Bhise)

Out of the 8 students, 5 are regular and 3 are defaulters. Isha Kulkarni attended 15 out of 20 lectures, which is exactly 75.0%, so she is REGULAR. Rohan Wankhede attended 14 out of 20 lectures, which is 70.0%, so he is a DEFAULTER. These two cases show that the 75% rule is applied correctly on the boundary value.

In the GUI version, the Students tab is used to add or delete students. In the Mark Attendance tab a date is selected and the status of a student is changed by double-click in the order Present, Absent, Late. The Dashboard shows the total students, present today, absent today and the number of defaulters. The Report tab shows Present, Absent, Late, Total Days, Attendance % and Status for every student. In Fig. 5.1.7 the report is shown before any attendance is counted, so all students have 0 days and the status is No Data.

The attendance in the demo mode is generated with the random module using a fixed seed, so it is only sample data and not the real attendance of any class. With real data the teacher marks P or A by hand and the same calculation is done.

Testing: all 18 automated tests passed (6 percentage tests, 5 status tests and 7 input validation tests). The tests include the boundary values 74.9%, 75% and 0 lectures held. Wrong inputs such as "x" and an empty input return None and the teacher is asked again, therefore the program does not crash.

Raw input	Normalized value
'p'	P
'P'	P
' present '	P
'a'	A
'ABSENT'	A
'x' (invalid)	None, ask again
'' (empty)	None, ask again

Present / Total	Percentage	Status
20 / 20	100.0%	REGULAR
15 / 20	75.0%	REGULAR
14 / 20	70.0%	DEFAULTER
10 / 20	50.0%	DEFAULTER
0 / 20	0.0%	DEFAULTER

5.3 APPLICATION

- It can be used by a teacher or a class representative to mark and check the attendance of a class quickly.
- It helps to find the defaulter students early, so that they can be warned before the examination.
- It can be used by students to learn the basic concepts of Python in a practical and interesting way.
- The same logic can be used as a base for a GUI or web based attendance system.
- The GUI version shows how a simple desktop application can be made with Python.

CHAPTER 6
CONCLUSION

In this project a Student Attendance Tracker was developed in Python. The Phase I design was converted into a working program, and in the advanced version input validation was added so that wrong input does not crash the program. The program marks the attendance of every student for every lecture, calculates the attendance percentage, identifies the defaulters with the 75% rule and displays a clear report. An automated test suite of 18 tests checks the correctness every time before the program starts, and the class summary gives the average, highest and lowest attendance. In Phase III the Tkinter GUI makes marking the attendance and viewing the report easy.

Through this project the basic concepts of Python like variables, input and output, conditional statements, loops, lists, dictionaries, tuples, functions and testing were practised. The whole data is handled only with variables and data structures, without any file or database.

FUTURE SCOPE

- To save the attendance in a file or a database so that it is not lost when the program is closed.
- To keep the attendance subject-wise and date-wise for a full semester.
- To improve the GUI with date-wise reports, subject-wise attendance and search of students.
- To export the report as CSV or PDF and to send a warning message to the defaulter students.
- To add a login for the teacher and to package the program as a standalone application.

REFERENCES

[1] Python Software Foundation, "random - Generate pseudo-random numbers", Python 3 Documentation. https://docs.python.org/3/library/random.html

[2] Python Software Foundation, "Built-in Types: dict, list and tuple", Python 3 Documentation. https://docs.python.org/3/library/stdtypes.html

[3] Python Software Foundation, "tkinter - Python interface to Tcl/Tk", Python 3 Documentation. https://docs.python.org/3/library/tkinter.html

[4] Allen B. Downey, *Think Python: How to Think Like a Computer Scientist*, 2nd Edition, O'Reilly Media, 2015.

[5] Al Sweigart, *Automate the Boring Stuff with Python*, 2nd Edition, No Starch Press, 2019.